

Lab 2 IAPP

Problemas del 7 al 10

```
1 perro(firulais).
2 perro(bruno).
3 perro(max).
4 gato(misu).
5 gato(luna).
6 gato(chanel).
7 gato(orion).
8 ave(piolin).
9 dueno(ana, firulais).
10 dueno(ana, misu).
11 dueno(luis, luna).
12 dueno(luis, orion).
13 dueno(luis, firulais).
14 dueno(maria, piolin).
15 dueno(julia, chanel).
16 dueno(pedro, bruno).
17
18 %Ejercicio 7
19 persona(Persona) :- dueno(Persona, _).
20 tiene_mascota(Persona) :- dueno(Persona, _).
21 todos_tienen_mascota :- forall(persona(Persona), tiene_mascota(Persona)).
22 %Ejercicio 8
23 mascota(M) :- perro(M).
24 mascota(M) :- gato(M).
25 mascota(M) :- ave(M).
26 todas_las_mascotas_tienen_dueno :- forall(mascota(M), dueno(_, M)).
27 %Ejercicio 9
28 amante_animales(Persona) :- dueno(Persona, Perro), perro(Perro), dueno(Persona, Gato), gato(Gato).
29 amantes_animales(Personas) :- setof(P, amante_animales(P), Personas).
30 amantes_findall(Personas) :- findall(P, amante_animales(P), Lista), sort(Lista, Personas).
31 amantes_setof(Personas) :- setof(P, amante_animales(P), Personas).
32 %Ejercicio 10
33 mascota_compartida(Persona1, Persona2, Mascota) :- dueno(Persona1, Mascota), dueno(Persona2, Mascota), Persona1 \= Persona2.
34 mascotas_compartidas(Persona1, Persona2, Mascotas) :- setof(M, mascota_compartida(Persona1, Persona2, M), Mascotas).
35
```

todos_tienen_mascota.

true

todas_las_mascotas_tienen_dueno.

false

amantes_animales(Personas).

Personas = [ana, luis]

mascotas_compartidas(ana, luis, Mascotas).

Mascotas = [firulais]

?- mascotas_compartidas(ana, luis, Mascotas).

Problema rana

```
%Problema de la rana
ubicacion(orilla_inicial, 0, 5).
ubicacion(piedra1, 2, 4).
ubicacion(piedra2, 5, 6).
ubicacion(piedra3, 8, 4).
ubicacion(piedra4, 5, 0).
ubicacion(orilla_final, 10, 5).
salto_maximo(4.0).

distancia(X1, Y1, X2, Y2, D) :- D is sqrt((X1 - X2)^2 + (Y1 - Y2)^2).
siguiente_estado(pos(LugarActual), pos(LugarSiguiente)) :- ubicacion(LugarActual, X1, Y1), ubicacion(LugarSiguiente, X2, Y2), LugarActual \= LugarSiguiente, salto_maximo(Max), distancia(X1, Y1, X2, Y2, D), D <= Max.
dfs(pos(orilla_final), _, [pos(orilla_final)]).
dfs(EstadoActual, Visitados, [EstadoActual | Camino]) :- siguiente_estado(EstadoActual, EstadoSiguiente), \+ member(EstadoSiguiente, Visitados), dfs(EstadoSiguiente, [EstadoSiguiente | Visitados], Camino).
buscar_solucion(Solucion) :- EstadoInicial = pos(orilla_inicial), dfs(EstadoInicial, [EstadoInicial], Solucion).
```

 `buscar_solucion(Solucion).`

`Solucion = [pos(orilla_inicial), pos(piedra1), pos(piedra2), pos(piedra3), pos(orilla_final)]`
`false`

`?- buscar_solucion(Solucion).`